SVKM's NMIMS

School of Technology Management & Engineering, Indore

Software Development Project Report

CampusFix

Infrastructure Complaint and Maintenance Management System

Course: Software Engineering

Program: MBA Tech. (Computer Engineering)

Year III, Semester V | Academic Year 2026-27

Submitted by

Name	Roll No.	SAP ID
Gurmann Singh Chiraunde	H037	70472400408
Disha Singh Kushwaha	H001	70472300203
Mahima Singh	B014	70472300260

Date of submission: 4 October 2026

Table of Contents

Section	Page
1. Introduction and Problem Statement	**1**
1.1 Background	1
1.2 Existing System	1
1.3 Problems with the Existing System	1
1.4 Problem Statement	1
2. Proposed System and Scope	**1**
2.1 Users and Roles	2
2.2 Complaint Categories and Priority Deadlines	2
2.3 Scope	2
3. User Stories	**2**
4. Use Case Model	**4**
4.1 Use Case Diagram	4
4.2 Use Case Descriptions	4
5. Activity Diagrams	**7**
5.1 Raise and Assign Complaint	7
5.2 Resolve and Confirm Complaint	7
5.3 Deadline Monitoring and Escalation	9
6. Class Diagram	**10**
7. Sequence Diagrams	**11**
8. State Transition Diagram	**12**
9. System Architecture	**14**
10. Conclusion	**14**
11. Learning Outcomes	**15**
12. References	**15**

1. Introduction and Problem Statement

1.1 Background

The NMIMS Indore campus has hostel rooms, academic blocks with classrooms and laboratories, a mess and canteen, and shared washrooms and common areas. Equipment in these spaces fails regularly: ceiling fans stop working, tube lights die, classroom ACs and projectors develop faults, chairs break and taps leak. Students and faculty are usually the first to notice these problems, but the campus has no single place where they can be reported and followed up.

1.2 Existing System

Complaints currently reach the maintenance team through a mix of informal channels. Some are told verbally to a warden, caretaker or admin staff member, while others are passed on through phone calls, WhatsApp messages or email. None of these channels is recorded centrally, so there is no history of what was reported, when, or what happened to it.

1.3 Problems with the Existing System

- **No record.** A verbal complaint can be forgotten, and the person who raised it has no reference number to follow up with.
- **No visibility.** The reporter cannot tell whether the complaint was received, assigned to someone, or is being worked on.
- **No prioritisation.** A sparking switchboard and a wobbly chair sit in the same informal queue. What gets fixed first depends on who follows up hardest rather than on urgency.
- **Closure without confirmation.** Work can be treated as done while the fault still exists, and the reporter has no way to say so.
- **No data for administrators.** There is no way to measure how long repairs take, which staff member is overloaded, or how satisfied users are with the service.

1.4 Problem Statement

To develop a centralized web platform for managing college infrastructure complaints and maintenance requests at NMIMS Indore, which allows students and faculty to report issues against an exact category and location, lets maintenance staff manage and resolve the requests assigned to them, and enables supervisors and administrators to monitor resolution time, staff workload, priorities and service quality. The system must close a complaint only after the reporter confirms the fix, and must escalate complaints automatically when they miss their resolution deadline.

2. Proposed System and Scope

CampusFix is a web platform where students and faculty raise complaints against a category and an exact location (block, floor and room). Every complaint receives a priority and a resolution deadline. A supervisor assigns it to a maintenance staff member with the right trade, and the reporter can track its status at every step. Two features set CampusFix apart from a plain complaint register:

- **Reporter-confirmed closure.** When staff mark a complaint resolved, the reporter is asked to confirm the fix and rate the service. A rejected fix reopens the complaint. If the reporter does not respond within 48 hours, the complaint closes automatically.
- **Priority deadlines with auto-escalation.** Each priority level has a resolution deadline. A complaint that misses it is escalated to the supervisor, and to the Administrator if it remains unresolved for a further 24 hours.

2.1 Users and Roles

User	Role in the system
Student / Faculty (Campus User)	Raise complaints, track their status, and confirm or reject fixes with a rating.
Maintenance Staff	Electricians, plumbers, carpenters, housekeeping and technicians. View assigned work and update its status.
Maintenance Supervisor	Review new complaints, set or adjust priority, assign staff, mark invalid or duplicate complaints, and handle escalations.
Administrator	Manage categories, locations, users and deadline rules, handle mess and canteen service complaints, and monitor performance.

2.2 Complaint Categories and Priority Deadlines

Infrastructure categories are Electrical, Plumbing, Furniture, HVAC (AC and fans), IT and AV equipment, Civil (walls, doors, ceilings) and Housekeeping. Mess and Canteen complaints form a separate Service category, because food quality and hygiene issues are not repairs; these are routed to the Administrator for action with the vendor. The default deadlines below can be changed by the Administrator.

Priority	Example	Resolve within
Critical	Sparking switchboard, water leaking near wiring	4 hours
High	Classroom AC not working in summer, no water supply	24 hours
Medium	Fan not working in a hostel room	48 hours
Low	Broken chair, peeling paint	5 days

2.3 Scope

In scope: complaint registration with category, location, description and optional photo; priority and deadline tracking; assignment to staff and status updates; reporter confirmation, rating and reopening; automatic escalation and auto-close; in-app notifications; role-based dashboards; and reports on resolution time, workload, priorities and service quality.

Out of scope: spare parts inventory and purchasing, vendor billing, a native mobile app (the web app is mobile-friendly instead), SMS or WhatsApp notifications, and single sign-on with existing college systems. QR codes on assets, merging of duplicate complaints and repeat-fault detection are listed as future enhancements.

3. User Stories

User stories are written in the standard format: As a <role>, I want <goal>, so that <benefit>. Each story has acceptance criteria that define when it is complete.

ID	User Story	Acceptance Criteria
US-01	As a hostel student, I want to report a broken fan by choosing the category and my exact room, so that the right staff member knows what to fix and where.	A complaint cannot be submitted without a category and location. A complaint ID is shown after submission.
US-02	As a student, I want to attach a photo to my complaint, so that staff can see the problem before they arrive.	JPG or PNG up to 5 MB. The photo is visible to the supervisor and the assigned staff member.

ID	User Story	Acceptance Criteria
US-03	As a campus user, I want to see the current status of my complaint, so that I do not have to follow up in person.	The full status history with the time of each change is visible to the reporter.
US-04	As a faculty member, I want safety issues such as a sparking switchboard to be handled first, so that dangerous faults are fixed before minor ones.	Complaints marked Critical get a 4-hour deadline and appear at the top of the supervisor's list.
US-05	As a maintenance supervisor, I want new complaints sorted by priority and due date, so that I can assign the urgent ones first.	The supervisor can change the priority, and the due date is recalculated when they do.
US-06	As a maintenance supervisor, I want to assign a complaint to an available staff member with the right trade, so that the work goes to someone who can do it.	Only available staff of the matching trade are listed. The assignee and the reporter are notified.
US-07	As a maintenance staff member, I want to update the status of my assigned complaints, so that everyone knows whether work is in progress or waiting for parts.	Staff can set In Progress, On Hold (with a reason) and Resolved (with remarks).
US-08	As a campus user, I want to confirm or reject a fix, so that a complaint is not closed while the problem still exists.	Rejecting needs a reason and reopens the complaint. Confirming needs a 1 to 5 rating. No response in 48 hours closes it automatically.
US-09	As a maintenance supervisor, I want overdue complaints escalated to me automatically, so that nothing is quietly forgotten.	When the due date passes, an escalation is recorded and the supervisor is notified. After 24 more hours, the Administrator is also notified.
US-10	As an administrator, I want a dashboard of resolution time, staff workload, deadline breaches and ratings, so that I can judge the quality of the maintenance service.	Figures can be filtered by category, block and date range.
US-11	As an administrator, I want mess and canteen complaints to come to me, so that food quality issues reach the vendor instead of the repair queue.	Complaints in the Service category are routed to the Administrator's queue.

4. Use Case Model

4.1 Use Case Diagram

The diagram shows six actors around the CampusFix system boundary. Student and Faculty specialise the general Campus User actor, so both inherit its use cases. Timer is a non-human actor, shown as a rectangle with the «actor» keyword, which triggers the time-based use cases. An «include» arrow points from the base use case to the behaviour it always uses, while an «extend» arrow points from the optional behaviour to the base use case at the named extension point.

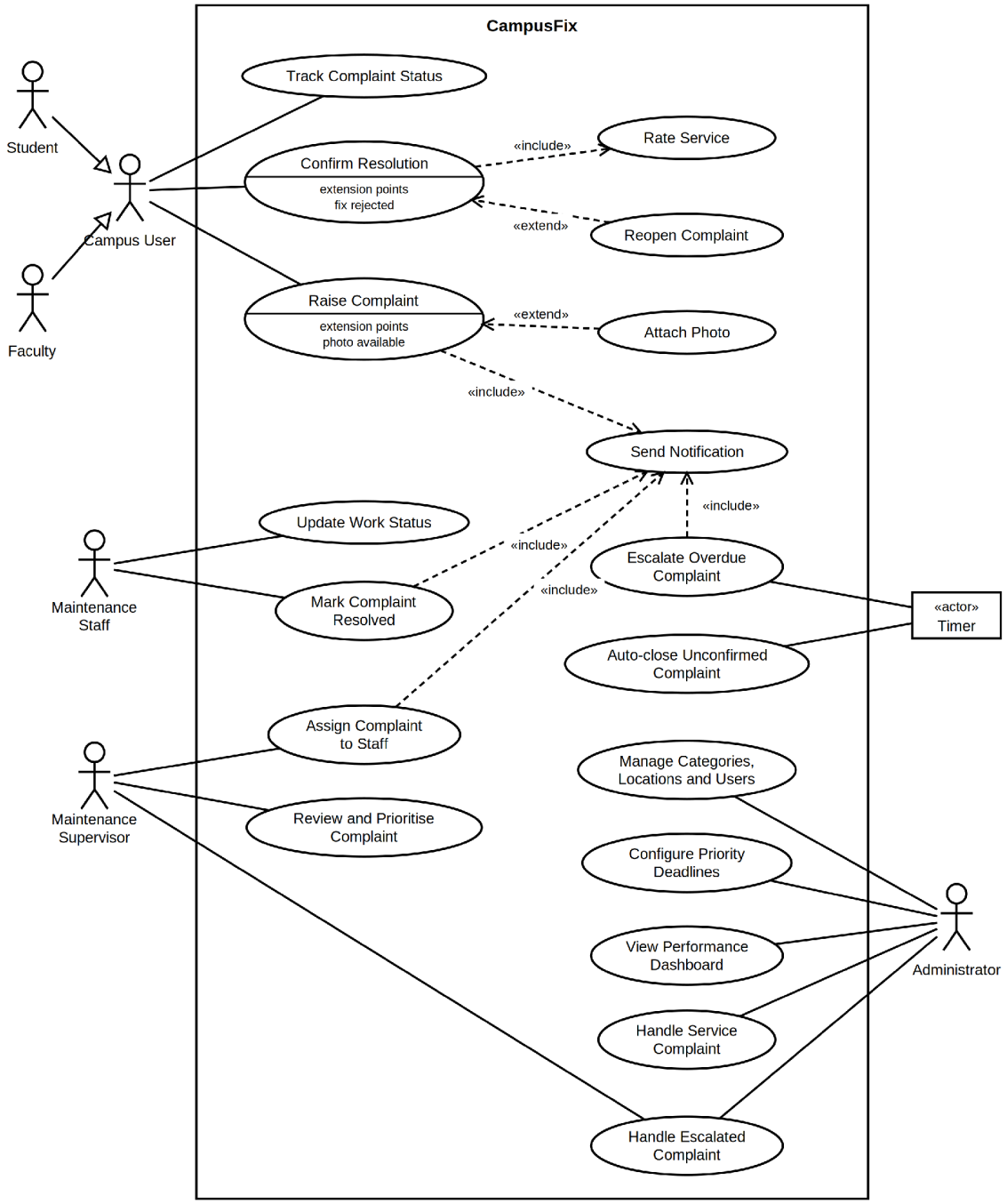


Figure 1: Use case diagram of CampusFix

4.2 Use Case Descriptions

UC-01: Raise Complaint

Use Case Name	Raise Complaint
ID	UC-01
Importance Level	High
Primary Actor	Campus User (Student or Faculty)
Use Case Type	Detail, essential
Stakeholders and Interests	Campus User: wants the fault fixed quickly. Supervisor: wants complete and accurate details. Administrator: wants every issue recorded.
Brief Description	A campus user reports an infrastructure or service problem against a category and exact location.
Trigger	The user notices a fault. Type: External.
Relationships	Association: Campus User. Include: Send Notification. Extend: Attach Photo. Generalization: Student and Faculty specialise Campus User.
Normal Flow of Events	1. The user opens the new complaint form. 2. The user selects a category. 3. The user selects the block, floor and room. 4. The user enters a description. 5. The user submits the complaint. 6. The system validates the details. 7. The system sets the priority from the category and calculates the due date. 8. The system saves the complaint with status Submitted and shows the complaint ID. 9. The system notifies the supervisor (Send Notification).
Alternate / Exceptional Flows	4a. If the user has a photo, Attach Photo extends this use case at the extension point photo available. 6a. If required details are missing, the system shows the errors and the flow returns to step 2. 9a. If the category type is Service, the Administrator is notified instead of the supervisor.

UC-04: Confirm Resolution

Use Case Name	Confirm Resolution
ID	UC-04
Importance Level	High
Primary Actor	Campus User (the reporter)
Use Case Type	Detail, essential
Stakeholders and Interests	Reporter: wants the fault actually fixed. Staff: want completed work recognised. Administrator: wants honest service ratings.
Brief Description	The reporter checks the work done and either confirms the fix with a rating or rejects it.
Trigger	Staff mark the complaint resolved and the system asks the reporter to confirm. Type: External.
Relationships	Association: Campus User. Include: Rate Service. Extend: Reopen Complaint.

Normal Flow of Events	1. The reporter opens the complaint from the notification. 2. The system shows the staff member's resolution remarks. 3. The reporter checks the fix in person. 4. The reporter selects Fixed. 5. The reporter rates the service from 1 to 5 and may add a comment (Rate Service). 6. The system records the feedback and sets the status to Closed. 7. The system records the total resolution time.
Alternate / Exceptional Flows	4a. If the reporter selects Not fixed, Reopen Complaint extends this use case at the extension point fix rejected: the reporter enters a reason, the status becomes Reopened, and the supervisor is notified. 1a. If the reporter does not respond within 48 hours, the Timer actor triggers Auto-close Unconfirmed Complaint instead.

UC-13: Escalate Overdue Complaint

Use Case Name	Escalate Overdue Complaint
ID	UC-13
Importance Level	High
Primary Actor	Timer
Use Case Type	Detail, essential
Stakeholders and Interests	Supervisor: wants to know about delays before users complain. Administrator: wants repeated delays to reach management.
Brief Description	The system checks open complaints on a schedule and escalates those that have passed their due date.
Trigger	The scheduled deadline check runs every 15 minutes. Type: Temporal.
Relationships	Association: Timer. Include: Send Notification.
Normal Flow of Events	1. The Timer starts the deadline check. 2. The system finds open complaints whose due date has passed. 3. For each overdue complaint, the system raises the escalation level and saves an escalation record. 4. The system sets the status to Escalated. 5. The system notifies the supervisor (Send Notification).
Alternate / Exceptional Flows	2a. If no complaint is overdue, the use case ends. 5a. If the escalation level is 2 or above, the Administrator is notified as well.

5. Activity Diagrams

The activity diagrams use swimlanes (partitions) to show which participant performs each action. Decision nodes carry a guard condition in square brackets on every outgoing flow, merge nodes rejoin the branches, and fork and join bars show actions that happen in parallel.

5.1 Raise and Assign Complaint

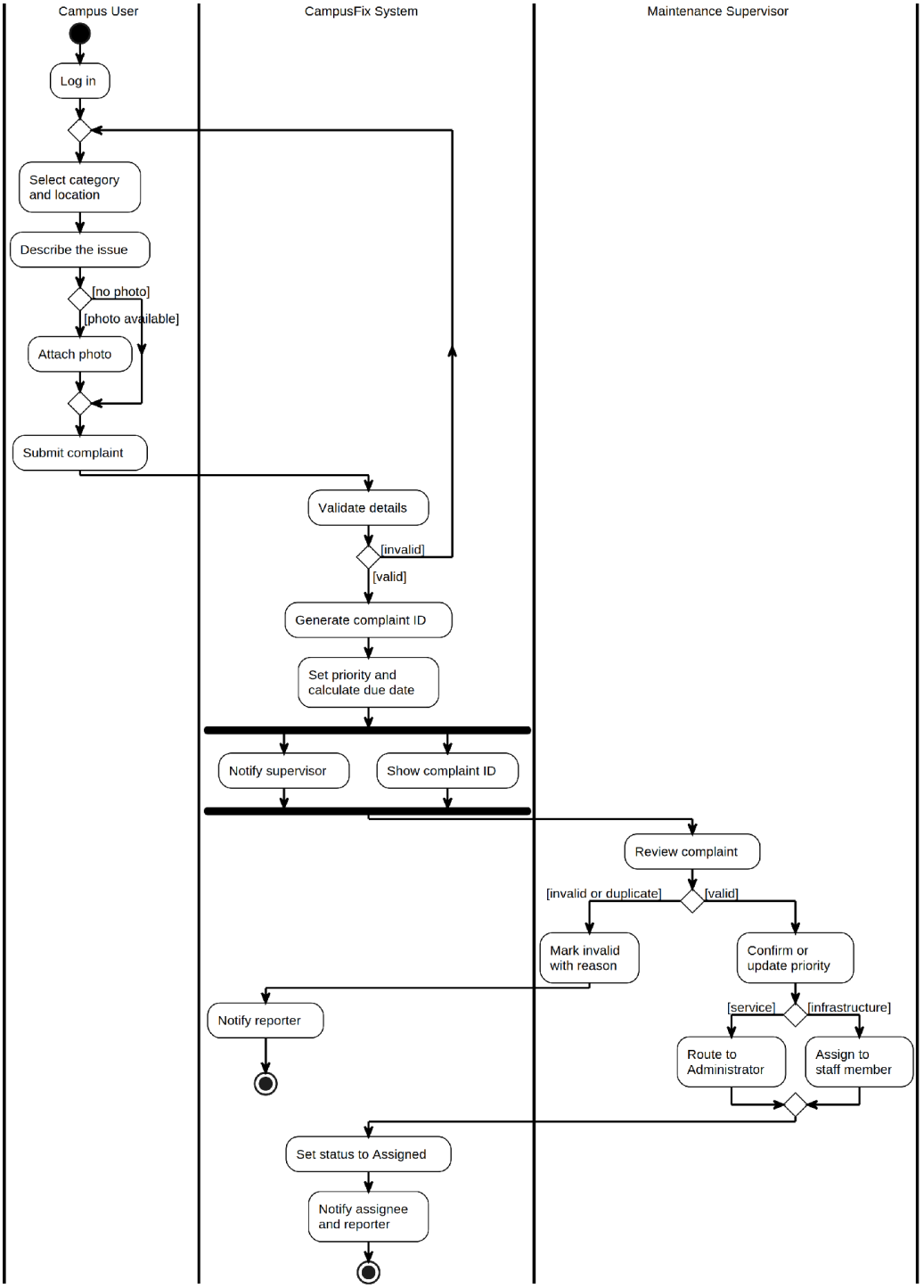


Figure 2: Activity diagram for raising and assigning a complaint

5.2 Resolve and Confirm Complaint

This workflow covers the work done by maintenance staff, including waiting for parts, and the reporter's decision. A rejected fix reopens the complaint and ends this activity, while both closing paths merge before the resolution time is recorded.

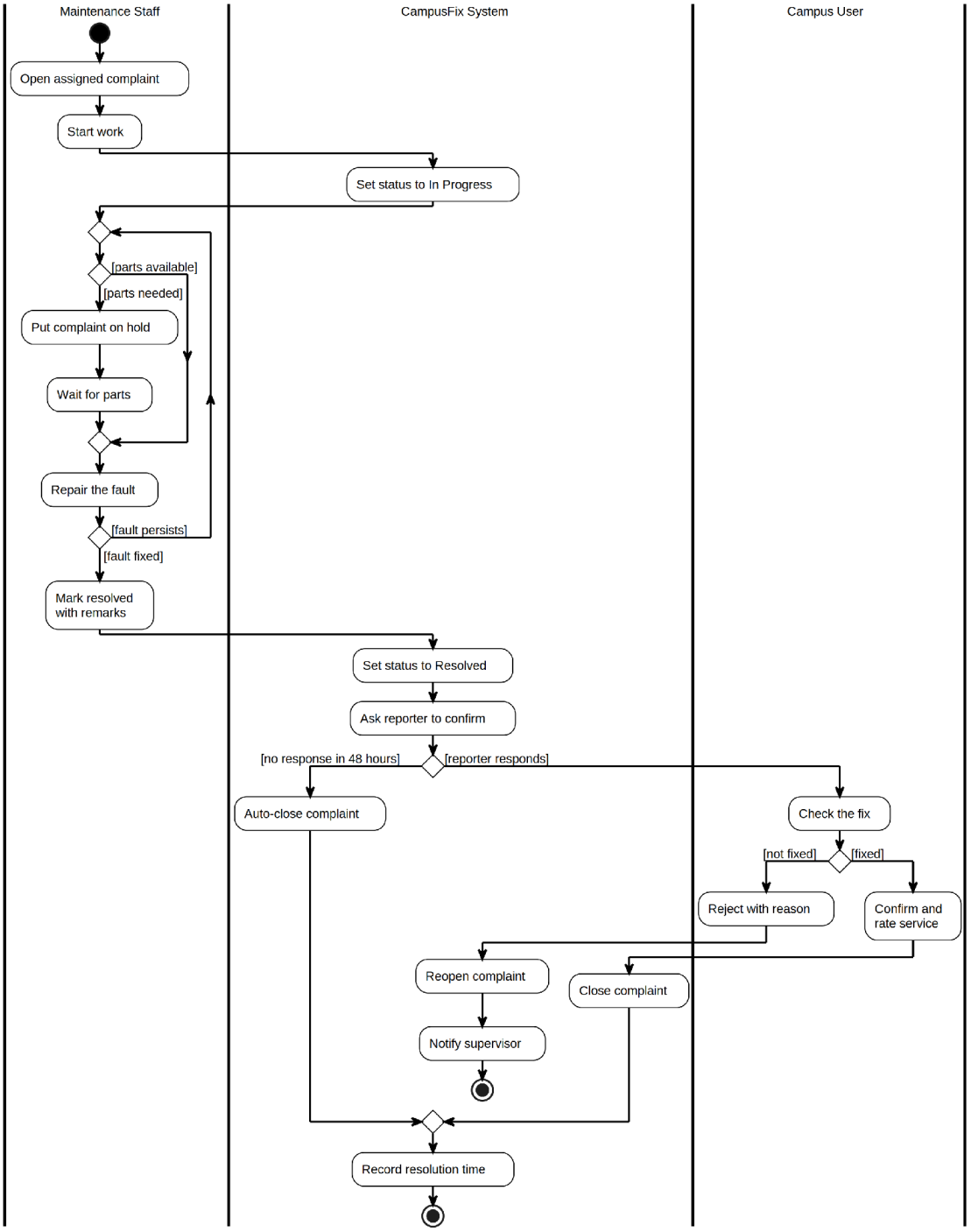


Figure 3: Activity diagram for resolving and confirming a complaint

5.3 Deadline Monitoring and Escalation

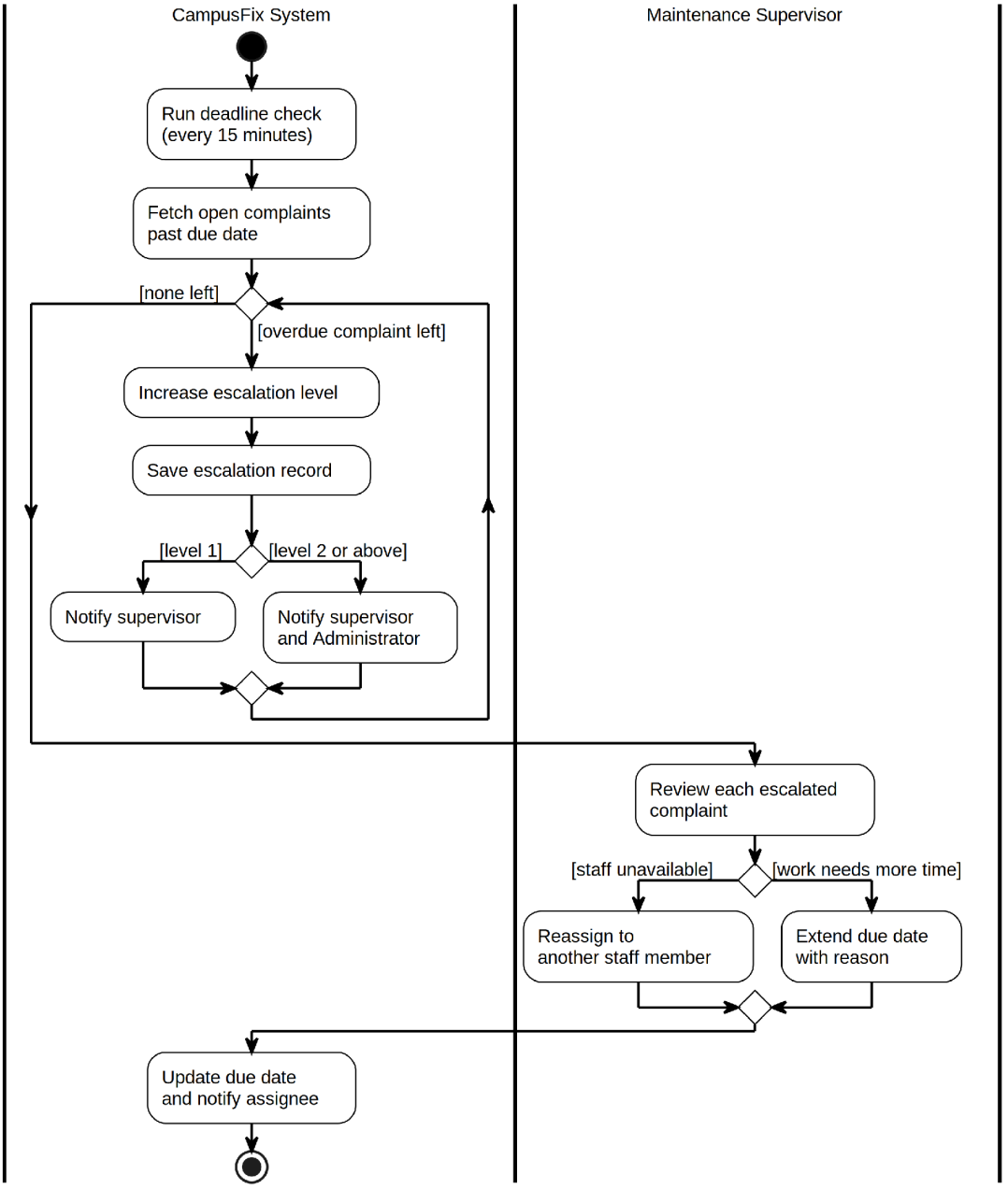


Figure 4: Activity diagram for deadline monitoring and escalation

6. Class Diagram

The class diagram shows the static structure of CampusFix. User and CampusUser are abstract classes, shown in italics. Attributes are private (-) and operations are public (+). The main relationships are described below the diagram.

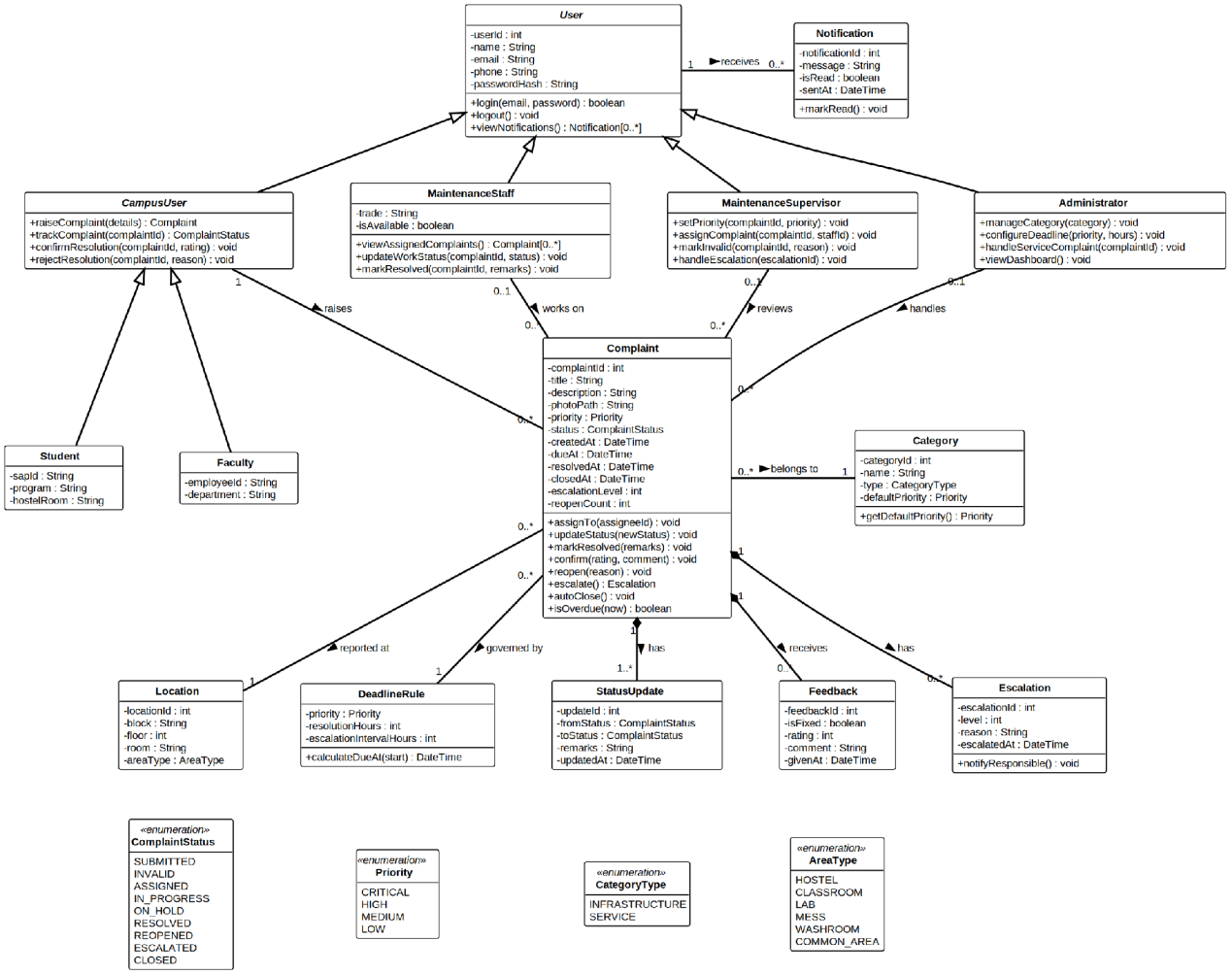


Figure 5: Class diagram of CampusFix

- **Generalization:** CampusUser, MaintenanceStaff, MaintenanceSupervisor and Administrator inherit from User; Student and Faculty inherit from CampusUser.
- **Association:** a CampusUser raises zero or more complaints; each Complaint belongs to exactly one Category, is reported at one Location and is governed by one DeadlineRule.
- **Composition:** StatusUpdate, Feedback and Escalation records cannot exist without their Complaint, so they are shown with a filled diamond on the Complaint side.

7. Sequence Diagrams

Following the use case realization approach, each sequence diagram uses one boundary object for the actor and use case pair (for example ComplaintForm), one controller object for the use case (for example ComplaintController), and entity objects taken from the class diagram. Synchronous calls are solid arrows, returns are dashed arrows, and activation bars show when an object is doing work. Boundary, controller and service objects are design-level objects and are therefore not part of the domain class diagram.

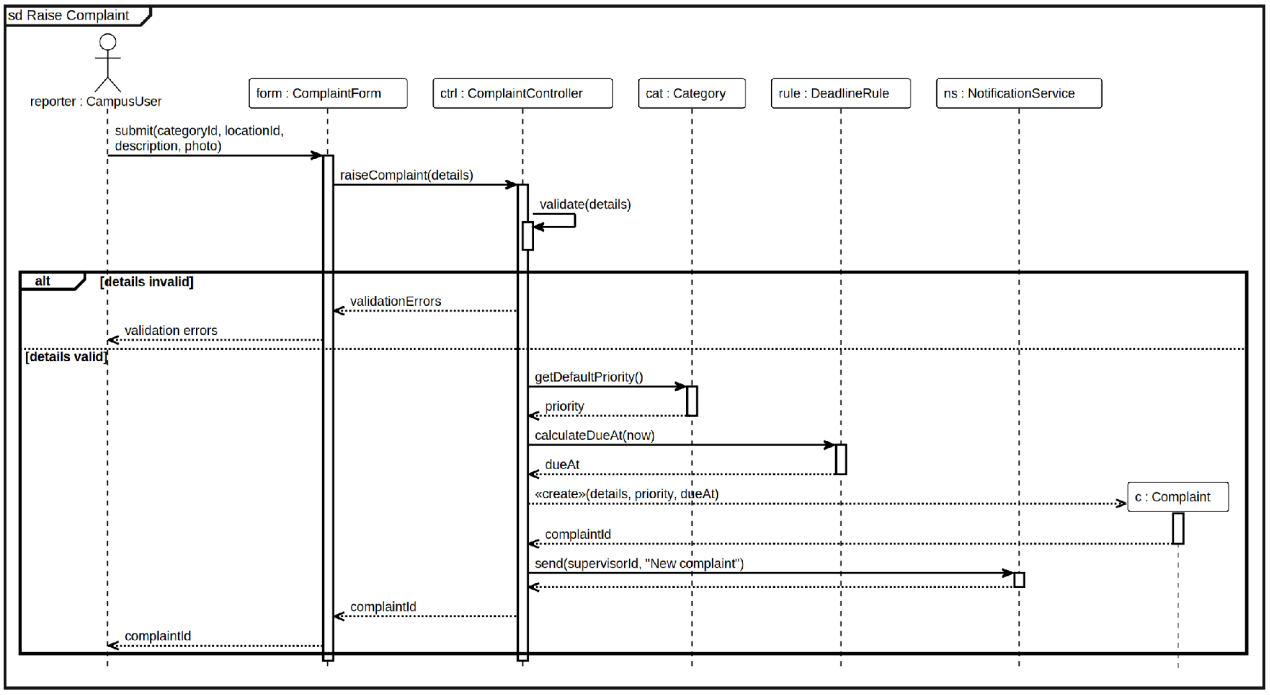


Figure 6: Sequence diagram for Raise Complaint

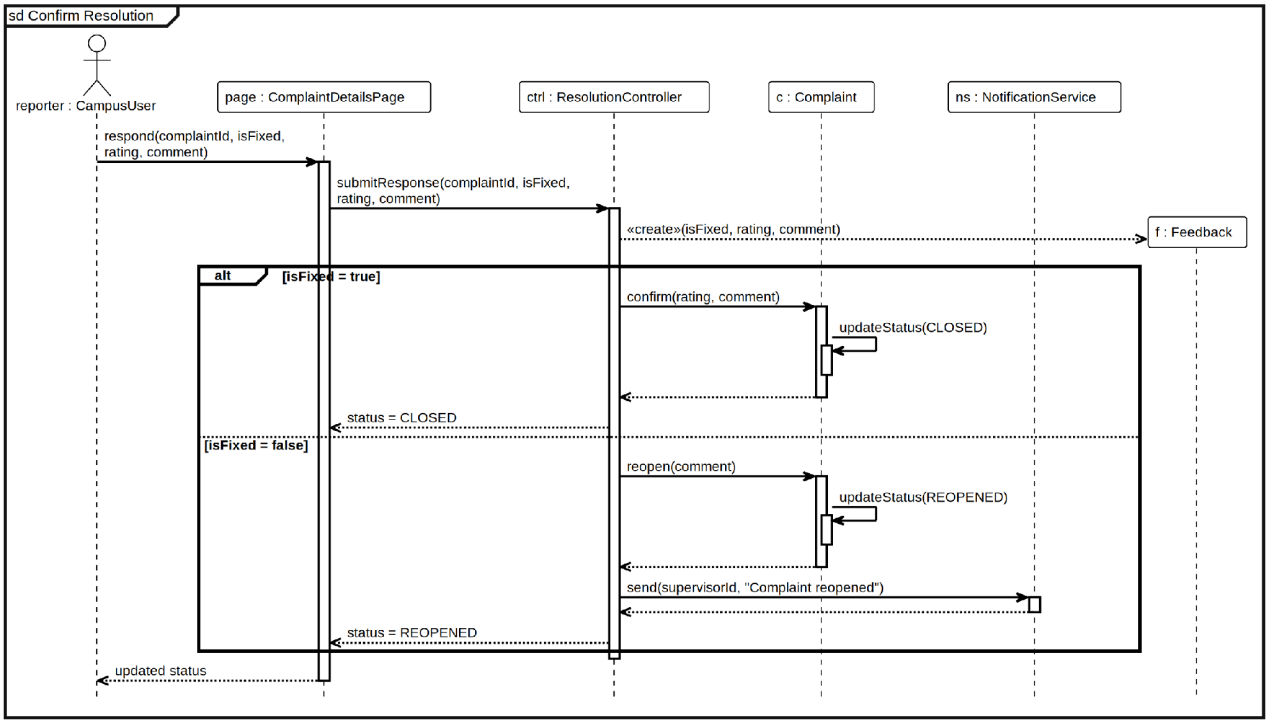


Figure 7: Sequence diagram for Confirm Resolution

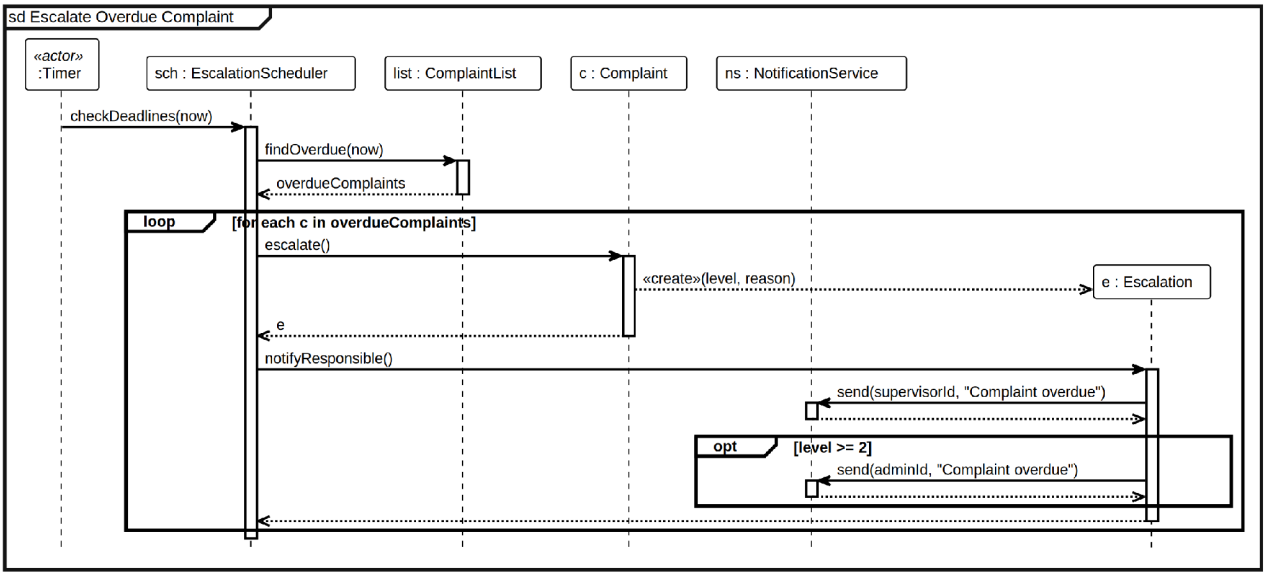


Figure 8: Sequence diagram for Escalate Overdue Complaint

8. State Transition Diagram

The Complaint class passes through a clear life cycle, so its behavioural state machine is shown on the next page, rotated to use the full length of the page. Open is a composite state containing Submitted, Assigned, In Progress, On Hold and Reopened; a single transition from its boundary moves any open complaint to Escalated when the deadline passes. Transitions are labelled event [guard] / action. The time events after(24 hours) and after(48 hours) and the change event when(now > dueAt) are fired by the passage of time rather than by a user.

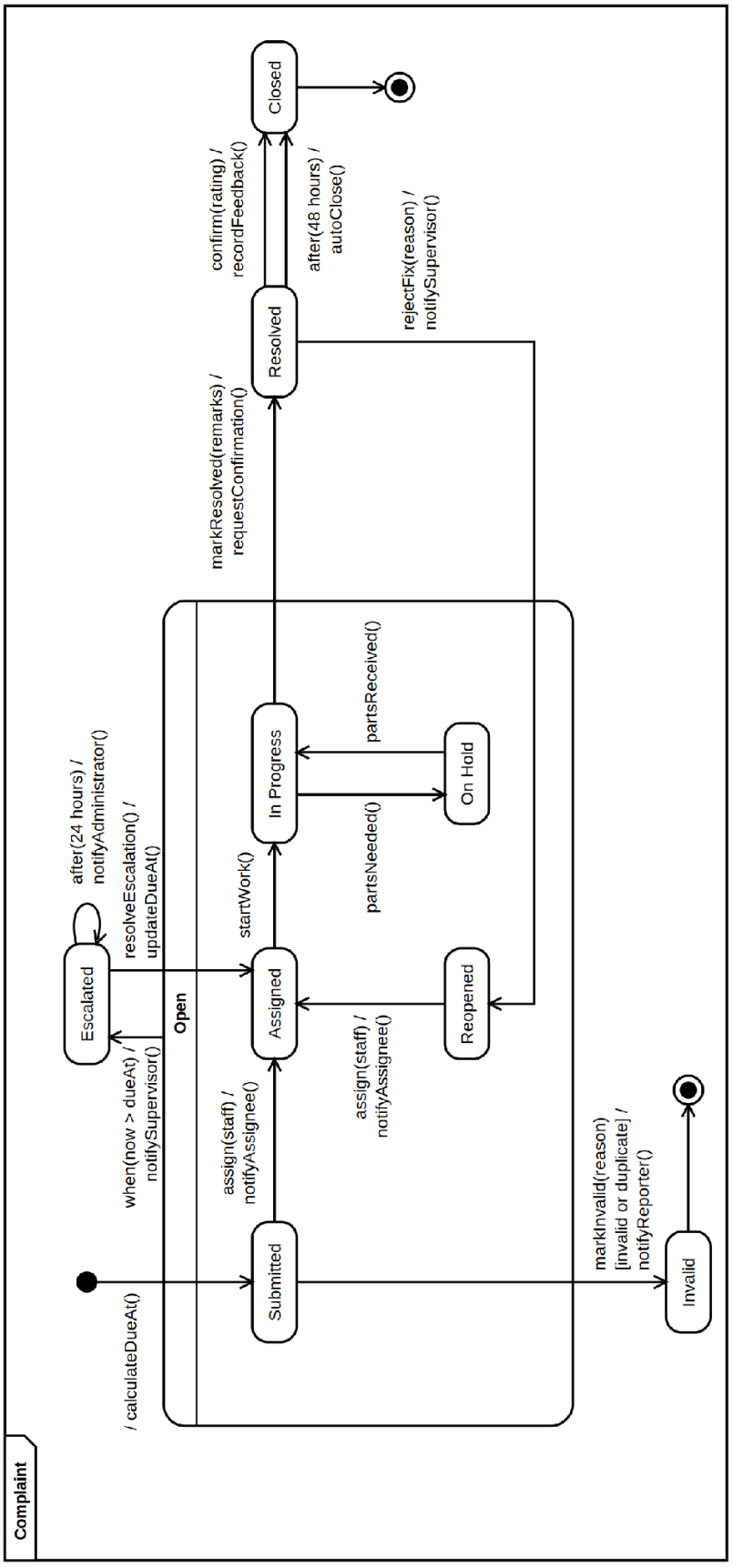


Figure 9: State transition diagram for a Complaint

9. System Architecture

CampusFix follows a layered architecture combined with the client-server model. The presentation layer runs in the browser and provides a separate portal for each role. The application layer exposes a REST API, applies authentication and role checks to every request, and contains the services that implement the business rules. The data layer is a single MySQL database at the centre of the system, which every service reads and writes, giving it the character of a data-centered architecture. Each layer talks only to the layer directly below it, so a change in one layer, such as replacing the user interface, does not affect the others.

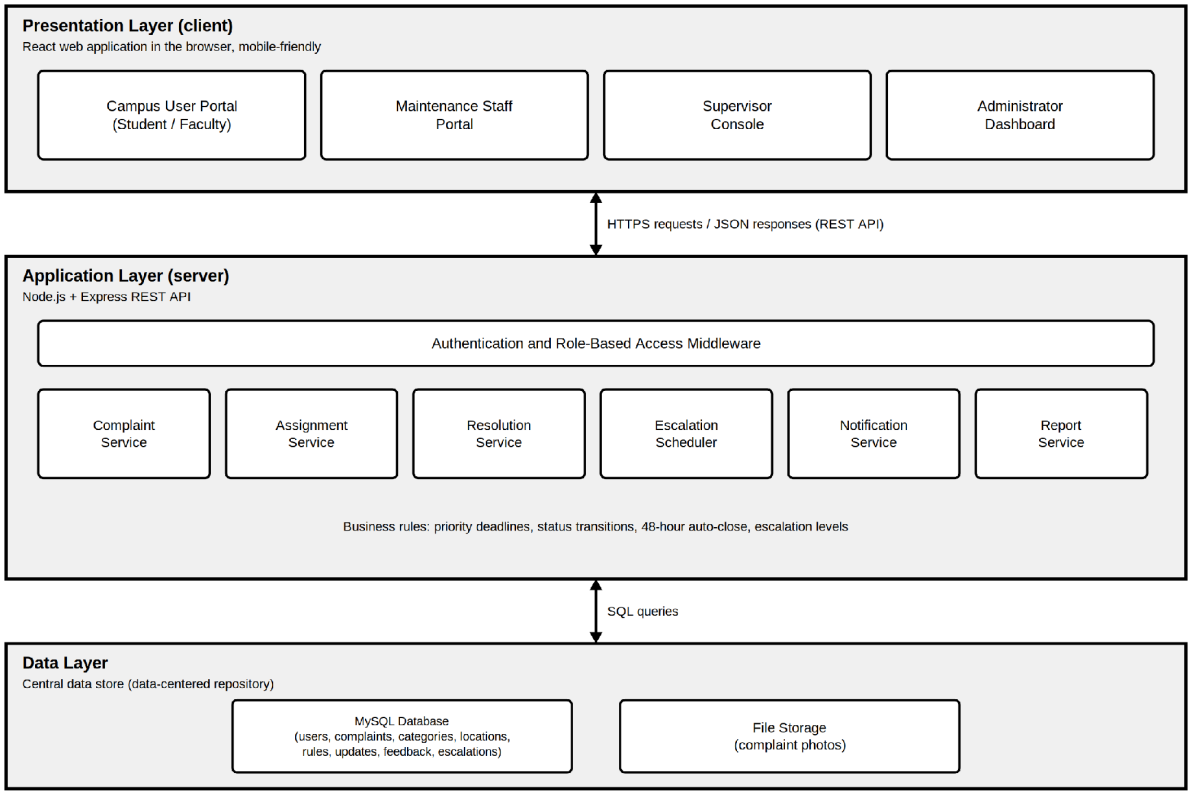


Figure 10: Layered system architecture of CampusFix

Layer	Technology	Responsibility
Presentation	React (web, mobile-friendly)	Role-based portals, forms, status tracking and dashboards
Application	Node.js with Express	REST API, authentication, business rules, scheduled deadline checks
Data	MySQL and file storage	Persistent records of users, complaints, history, feedback and photos

10. Conclusion

CampusFix replaces scattered verbal and WhatsApp complaints with one system in which every complaint has an owner, a deadline and a visible status. The design keeps the reporter involved until the fault is actually fixed, and the deadline and escalation rules make sure that delays reach someone who can act on them. The same records give administrators measurable data on resolution time, workload and service quality. The diagrams in this report describe the complete system, and the code demonstration will implement its core: complaint registration, assignment, status updates, reporter confirmation with reopening, and automatic escalation.

11. Learning Outcomes

- Writing user stories with acceptance criteria made each requirement testable instead of a loose wish.
- We learned the difference in direction between «include» and «extend»: the base use case points to what it always includes, while an optional extension points to the base at an extension point.
- Actor generalization (Student and Faculty under Campus User) and a non-human Timer actor let the use case diagram stay small while still covering time-triggered behaviour.
- Modelling the complaint life cycle showed how composite states, guards and time events express rules such as auto-close and escalation more clearly than text.
- Identifying boundary, controller and entity objects connected the use cases, the sequence diagrams and the class diagram, so method names stay consistent across the design.
- Choosing a layered, data-centered architecture showed how the architectural styles from the syllabus apply to a real campus system.

12. References

[1] Sommerville, I., Software Engineering, 10th ed., Pearson, 2016.

[2] Pressman, R. S., Software Engineering: A Practitioner's Approach, McGraw-Hill.

[3] Dennis, A., Wixom, B. H. and Tegarden, D., Systems Analysis and Design: An Object-Oriented Approach with UML, Wiley.

[4] Object Management Group, OMG Unified Modeling Language (OMG UML), Version 2.5.1, December 2017.

[5] Course notes, Software Engineering, Units 1 to 5, SVKM's NMIMS School of Technology Management & Engineering, Indore.